

1. What is Learning in AI?

- Learning means improving performance on a task based on experience (observations) rather than being explicitly programmed.
- The agent learns from data generated by the environment and generalizes to new, unseen situations.

Key Idea: A learning algorithm builds a model from observations and uses it to make predictions or decisions.

Example: Learn to predict whether an email is spam based on examples of labeled emails.

4. Types of Learning from Observations

(a) Supervised Learning

- Learn a mapping from inputs to outputs using labeled examples.
- Used for classification and regression.
- Examples: spam detection, image classification, house price prediction.

(b) Unsupervised Learning

- Find structure in unlabeled data.
- Examples: clustering, dimensionality reduction, density estimation.

(c) Reinforcement Learning (from interaction)

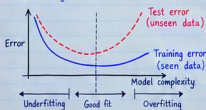
- Learn from rewards and experience with the environment.
- Covered in later chapters (Ch. 21).

(d) Other settings

- Semi-supervised learning: few labeled examples, many unlabeled examples.
- Active learning: choose informative examples.
- Online learning: learn continuously from a stream of data.

10. Generalization

- A good hypothesis should perform well not only on the training data, but also on new, unseen instances.
- Generalization is the ability to apply what was learned to new situations.
- A hypothesis that fits the training data perfectly may still generalize poorly (overfitting).



2. Components of a Learning Problem

Task (T) : what is to be learned (e.g., classification).
Experience (E) : data/observations used for learning.
Performance measure (P) : how success is evaluated (e.g., accuracy).

A program learns from experience E with respect to task T and performance measure P if its performance at T, as measured by P, improves with experience E.

5. Supervised Learning Basics

- Given a set of training examples $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ where x are input features and y are target labels.
- Learn a function $h(x)$ that predicts y for new instances x .
- Two main types:

• **Classification**: predict a discrete label (e.g., spam or not spam).

• **Regression**: predict a continuous value (e.g., house price).

7. Hypothesis Space

- A hypothesis is a candidate model h from a hypothesis space H .
- H is the set of all possible functions consistent with the input representation.
- Learning searches for the best hypothesis $h \in H$ using the training data.

Example: For boolean concepts over two binary features (x_1, x_2) , H could include all $2^{2^2} = 16$ possible functions.

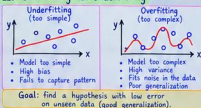
8. Example: Simple Hypothesis Space

Boolean functions of two binary features

Hypothesis h	x_1	x_2	$h(x_1, x_2)$
$h_1: x_1$	0	0	0
	1	0	1
	0	1	0
	1	1	1
$h_2: x_1 \text{ AND } x_2$	0	0	0
	1	0	0
	0	1	0
	1	1	1

Many hypotheses are possible. The learning algorithm searches H for a good one.

11. Overfitting vs. Underfitting



3. General Learning Framework



The learned model is used to make predictions or decisions for new, unseen inputs.

6. Concept Learning Overview

- Concept learning is a form of supervised learning where the target is a discrete concept (boolean function).
- Given labeled examples, learn a hypothesis that classifies instances as positive (+) or negative (-).

Example: Learn the concept "PlayTennis" from weather data.

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rain	Mild	High	False	Yes
Rain	Cool	Normal	False	Yes

9. Training Data and Examples

- Training examples consist of input-output pairs (x, y) drawn from an unknown target function f (the true concept).
- The examples are assumed to be independent and identically distributed (i.i.d.) from some distribution D .
- The more representative the training data, the better the learned hypothesis will generalize.

Note: The algorithm does not see the true function f , only a finite set of examples.

12. Key Takeaways

- Learning improves performance from experience rather than explicit programming.
- A learning problem is defined by task (T), experience (E), and performance measure (P).
- From observations we have supervised, unsupervised, and reinforcement learning.
- Supervised learning uses labeled examples to learn a mapping from inputs to outputs.
- Concept learning predicts discrete labels.
- A hypothesis space contains all possible candidate models.
- Good generalization to new instances is essential; avoid overfitting and underfitting.
- The quality and representativeness of training data strongly affect learning success.

1. Concept Learning as Search

- Concept learning can be viewed as searching through a hypothesis space of possible hypotheses.
- Each hypothesis $h \in H$ is a candidate description of the target function f .
- Use training examples to eliminate inconsistent hypotheses and find those that fit the data.
- The goal is to find a hypothesis that generalizes well to unseen instances.

Key Idea: Learning is a search problem in the space of hypotheses, using training examples to constrain the search.

4. Hypothesis Space for EnjoySport

- Each attribute has a finite set of possible values (plus '?').
- Hypothesis space H is the set of all conjunctive hypotheses over the attributes.
- Size of $H = \prod_{i=1}^n (|V_i| + 1)$.
- For EnjoySport (6 attributes):
 $|H| = 5 \times 5 \times 4 \times 4 \times 3 \times 3 = 3600$

Note: The hypothesis space can be very large, so we need efficient search methods such as Find-S and Candidate Elimination.

7. Candidate Elimination Overview

- Maintains a set of all hypotheses consistent with the training examples (version space).
- Represent the version space by two boundaries:
 - S : the set of most specific hypotheses in the version space.
 - G : the set of most general hypotheses in the version space.
- Starts with $S = \{\langle \emptyset, \dots, \emptyset \rangle\}$ and $G = \{\langle ?, \dots, ? \rangle\}$.
- For each training example (positive or negative), update S and G to remove inconsistent hypotheses.
- Always maintains S and G such that every hypothesis in S is more specific than every hypothesis in G .

10. Updating the Boundaries (Intuition)

Positive example x ($h(x) = 1$)

- Remove from G any hypothesis that does not cover x .
- For each $s \in S$ that does not cover x , generalize s to minimally cover x .
- Remove more general hypotheses from S (keep only most specific).

Negative example x ($h(x) = 0$)

- Remove from S any hypothesis that covers x .
- For each $g \in G$ that covers x , specialize g to minimally exclude x .
- Remove more specific hypotheses from G (keep only most general).

2. The EnjoySport Example

Target concept: EnjoySport(x)

Attributes: Sky, AirTemp, Humidity, Wind, Water, Forecast

Day	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

- Each instance is described by 6 attribute values.
- The target function EnjoySport(x) returns Yes/No.
- We assume the examples are noise-free.
- There are many possible hypotheses consistent with these examples.

5. The Find-S Algorithm

- Finds the most specific hypothesis consistent with the positive training examples.
- Ignores negative examples.
- Starts with the most specific hypothesis h_0 .
- For each positive example, generalize h to be minimally more general to cover the example.
- Result is the most specific hypothesis that is consistent with all positive examples.

Algorithm (Find-S)

- $h \leftarrow \langle \emptyset, \emptyset, \dots, \emptyset \rangle$
- For each positive training example x :
For each attribute i :
if $h[i] = \emptyset$ then $h[i] \leftarrow x[i]$
else if $h[i] \neq x[i]$ then $h[i] \leftarrow ?$
- Return h

8. The Version Space

- Version space = set of all hypotheses in H consistent with the training examples D .
- Represented compactly by boundaries S and G instead of explicitly listing all hypotheses.
- All hypotheses h in the version space satisfy:
 $\exists s \in S, g \in G$ such that $s \leq h \leq g$.



Note: S and G compactly represent a (possibly very large) set of consistent hypotheses.

3. Conjunctive Hypothesis Representation

- We use conjunctions of attribute constraints.
- Each attribute can be a specific value or '?' (any value).
- A hypothesis is a vector of attribute values.

Example: $h = \langle \text{Sunny, Warm, ?, Strong, ?, ?} \rangle$
means: Sky = Sunny AND AirTemp = Warm AND Wind = Strong (others can be anything).

Most specific hypothesis:

- $h_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$
- No instance is covered.
 - More specific than any other hypothesis.

Most general hypothesis:

- $h_g = \langle ?, ?, ?, ?, ?, ? \rangle$
- Covers all instances.
 - More general than any other hypothesis.

6. Running Find-S on EnjoySport

Step	Training example (positive)	h
0	(initial)	$\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$
1	$\langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle$	$\langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle$
2	$\langle \text{Sunny, Warm, High, Strong, Warm, Same} \rangle$	$\langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle$
3	$\langle \text{Sunny, Warm, High, Strong, Cool, Change} \rangle$	$\langle \text{Sunny, Warm, ?, Strong, ?, ?} \rangle$

Final hypothesis:

$h_{\text{Find-S}} = \langle \text{Sunny, Warm, ?, Strong, ?, ?} \rangle$

- Most specific consistent with positive examples.
- Ignores the negative example (Day 3).

9. Consistency with Training Examples

- A hypothesis h is consistent with a positive example x if $h(x) = 1$ (covers x).
- A hypothesis h is consistent with a negative example x if $h(x) = 0$ (does not cover x).
- More specific hypotheses cover fewer instances.
- More general hypotheses cover more instances.

Example: Consider $h = \langle \text{Sunny, Warm, ?, ?, ?} \rangle$

- Covers Day 1 (positive) $\rightarrow$ consistent
- Covers Day 2 (positive) $\rightarrow$ consistent
- Does not cover Day 3 (negative) $\rightarrow$ consistent
- Covers Day 4 (positive) $\rightarrow$ consistent

11. Key Takeaways

- Concept learning is searching in a hypothesis space.
- Use conjunctive hypotheses with attribute values or '?'.
- Find-S finds the most specific hypothesis consistent with positive examples (ignores negatives).
- Candidate elimination maintains the version space using specific boundary S and general boundary G .
- The version space contains all hypotheses consistent with the training examples.
- S and G are updated differently for positive and negative examples.
- S becomes more general and G becomes more specific as more examples are observed.

"Learning from observations searches the space of hypotheses to find those consistent with the data."

1. Candidate Elimination Algorithm

- Maintains a version space, represented by two boundaries:
- S : set of maximally specific hypotheses consistent with data.
- G : set of maximally general hypotheses consistent with data.
- On each training example, update S and G to keep only hypotheses consistent with all observed examples.

Key Idea: The true target concept is somewhere between S and G in the version space.

2. Updating S and G

(a) Positive example (x is Positive)

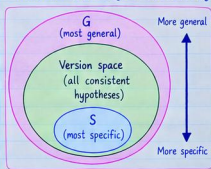
- Remove from G any hypothesis that does not classify x as positive.
- For each $s \in S$ that does not classify x as positive, remove it and add all its minimal generalizations that do classify x as positive (and are $\leq$ some g in G).
- Remove from S any hypothesis that is more general than another in S .

(b) Negative example (x is Negative)

- Remove from S any hypothesis that classifies x as positive.
- For each $g \in G$ that does classify x as positive, remove it and add all its minimal specializations that do not classify x as positive (and are $\geq$ some s in S).
- Remove from G any hypothesis that is more specific than another in G .

3. Version Space Boundaries

- S is the lower boundary (most specific) of the version space.
- G is the upper boundary (most general).
- The version space contains all hypotheses h such that $\exists s \in S, g \in G$ with $s \leq h \leq g$.



4. Worked Example: Candidate Elimination

Hypothesis space: EnjoySport with attributes (Outlook, Temperature, Humidity, Wind).

Values: {Sunny, Cloudy, Rain}, {Hot, Cool, High, Normal}, {Weak, Strong}.

#	Example (x)	y	S (after update)	G (after update)
0	(initial)	-	$\{\emptyset, \emptyset, \emptyset, \emptyset\}$	$\{?, ?, ?, ?\}$
1	(Sunny, Hot, High, Weak)	+	$\{(Sunny, Hot, High, Weak)\}$	$\{?, ?, ?, ?\}$
2	(Sunny, Cool, High, Strong)	+	$\{(Sunny, ?, High, ?)\}$	$\{?, ?, ?, ?\}$
3	(Rain, Cool, High, Strong)	-	$\{(Sunny, ?, High, ?)\}$	$\{(Sunny, ?, ?, ?)\}$
4	(Sunny, Cool, Normal, Strong)	+	$\{(Sunny, ?, ?, ?)\}$	$\{(Sunny, ?, ?, ?)\}$

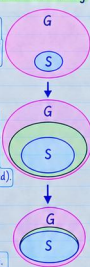
After 4 examples, $S = G = \{(Sunny, ?, ?, ?)\}$.
The target concept has been exactly identified.

5. Diagram: S and G Narrowing

Initially:
 S is most specific,
 G is most general.

After some positive examples:
 S generalizes,
 G is unchanged
(or slightly reduced).

After positive and negative examples:
 S and G converge.



6. Bias of the Hypothesis Representation

- The hypothesis space defines a bias: it restricts the set of concepts that can be learned.
- Candidate elimination can only find the target concept if it is representable in the hypothesis space H .

Example: If H contains only conjunctive hypotheses, we cannot learn a disjunctive target concept even with infinite data.

7. Inductive Bias

- Learning from limited data requires an inductive bias – assumptions that prefer some hypotheses over others.
- In candidate elimination, the bias is given by the chosen hypothesis space and the general-to-specific ordering.
- Different representations (e.g., conjunctions, disjunctions, decision lists) encode different biases and lead to different generalizations.

8. Why Find- S Works

- Find- S is a simplified version of candidate elimination that maintains only S .
- It works when the training data contains only positive examples and is noise-free.
- With only positive examples, S is generalized to cover all observed instances, and no negative evidence is needed.
- It returns the most specific hypothesis consistent with all observed positive examples.

If all training examples are positive and the target concept is in H , Find- S will converge to the target concept.

9. Noise-Free Assumption and Limitations

(a) Noise-Free Assumption

- Candidate elimination assumes the training data is consistent (noise-free).
- There exists at least one hypothesis in H that correctly classifies all training examples.
- In the presence of noise (misclassified examples), the version space can become empty.

(b) Limitations

- Sensitive to the choice of hypothesis space.
- Cannot handle noisy or contradictory data (without modification).
- May require many training examples to converge to a single hypothesis.
- Computationally expensive if H is very large.
- Does not provide a preference among hypotheses inside the version space.

10. Key Takeaways and Exam Tips

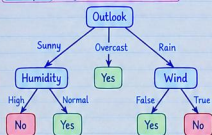
- Candidate elimination maintains S and G (boundaries of the version space).
- Update S and G differently for positive and negative examples.
- The version space contains all consistent hypotheses between S and G .
- The hypothesis representation defines the bias of learning.
- Find- S works for positive, noise-free data.
- Learning requires inductive bias due to limited data.
- Candidate elimination assumes noise-free data and has computational limits.
- Be able to trace a few examples and update S and G step by step.
- Understand how S and G converge.
- Common exam question: given a sequence of examples, compute S and G .

“The version space captures what we know and what we don't know.”

1. Decision Tree Learning Overview

- Learn a tree that tests attributes at internal nodes and predicts a class at leaves.
- Each path from root to leaf is a classification rule.
- Easy to understand, works for discrete attributes, handles multi-way tests.
- Learn recursively by choosing the best attribute at each step.

Example: PlayTennis decision tree



2. Attribute Selection Idea

- Choose the attribute that best separates the training examples into (mostly) pure subsets.
- Use a measure of impurity such as information gain.
- Recursively partition the data until the examples in a node are mostly of the same class or other stopping criteria.

Key Idea: Select the attribute that most reduces the uncertainty (impurity) in the class labels.

3. Information Gain and Entropy

- Entropy measures the impurity of a set of examples S .
- Entropy is low when the set is pure (all same class) and high when the set is mixed.

Entropy of a binary class set S :

$$H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

where p_+ and p_- are the fractions of positive and negative examples.

Information gain of attribute A :

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Choose the attribute with the highest information gain.

Information gain measures the expected reduction in entropy after splitting on A .

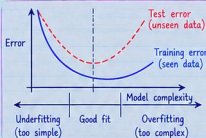
4. Training, Validation, and Test Sets

- Training set:** used to learn the model (e.g., grow a decision tree).
- Validation set:** used to tune parameters and choose the best model (e.g., tree size, pruning level).
- Test set:** used to estimate final performance on unseen data.

The test set should only be used after the model is completely chosen.

5. Generalization, Overfitting, Underfitting

- Generalization:** perform well on new, unseen examples.
- Overfitting:** model is too complex and fits noise in the training data.
- Underfitting:** model is too simple and fails to capture the true pattern.



Training error always decreases with model complexity, but test error is lowest at an intermediate complexity.

6. Bias-Variance Intuition

- Bias:** error from making strong assumptions (e.g., very simple models). High bias $\rightarrow$ underfitting.
- Variance:** error from being sensitive to the training data (e.g., very complex models). High variance $\rightarrow$ overfitting.
- Test error $\approx$ bias² + variance + irreducible error (noise in data).

As model complexity increases, bias decreases but variance increases.

7. Pruning Decision Trees

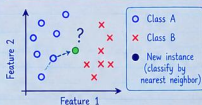
- Greedy tree growth can lead to overfitting.
- Pruning removes branches that do not improve generalization.
- Two main approaches:
 - Pre-pruning (early stopping).
 - Post-pruning (grow full tree, then prune).

Use a validation set or a complexity penalty to decide which branches to prune.

8. Nearest-Neighbor Learning

- To classify a new instance, find the k most similar training examples.
- Predict the most common class among the neighbors (majority vote).
- Simple and flexible; can handle complex decision boundaries.
- Requires a distance metric and feature representation.

Example: 1-Nearest Neighbor ($k=1$)



9. Explanation-Based Learning (EBL)

- Use a general domain theory to explain an observed training example.
- Generalize the explanation to create a more specific rule for similar situations.
- Especially useful when examples are rare but a theory is available.

Example: Given a domain theory of blocks and an example of a stack, derive a rule that generalizes to similar stacks.

10. Learning in Neural Networks (Preview)

- Neural networks learn by adjusting the weights of a network of simple units (neurons).
- Training uses many labeled examples and an error signal to update weights (e.g., backpropagation).
- Can learn complex, non-linear functions and good for perceptual tasks.
- Discussed in more detail in later chapters (Ch. 18-19).

Neural networks are another approach to learning from examples, with a different representation (weighted connections).

11. Summary and Key Takeaways

- Decision trees use attribute selection (e.g., information gain) to learn a tree.
- Use training, validation and test sets.
- Good generalization requires avoiding overfitting and underfitting.
- Bias-variance tradeoff explains the effect of model complexity.
- Pruning helps improve decision trees.
- Nearest neighbors classify by similarity.
- Explanation-based learning uses a domain theory to generalize from a few examples.
- Neural networks learn by adjusting weights and can model complex functions.

Summary and Revision

1. Key Definitions

- Training example:** A pair (x, y) where x is an input (instance) and y is the target output (label).
- Hypothesis:** A candidate function h from hypothesis space H that maps inputs to outputs.
- Concept:** The true (unknown) target function f that we want to learn.
- Hypothesis space (H):** The set of all possible hypotheses that the algorithm can represent.
- Version space:** The set of all hypotheses in H that are consistent with all observed training examples.
- Generalization:** The ability to perform well on new, unseen instances.
- Overfitting:** Learning a hypothesis that fits the training data very well but does not generalize to new instances.

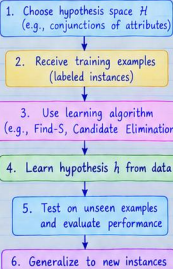
2. Types of Learning (Quick Comparison)

Type	Training Data	Goal	Examples
Supervised Learning	Labeled examples (x, y)	Learn a mapping $x \rightarrow y$	Classification, Regression (e.g., spam)
Unsupervised Learning	Unlabeled data (x)	Find structure in data	Clustering, Dimensionality reduction
Reinforcement Learning	Interaction with environment	Learn a policy to maximize cumulative reward	Game playing, Robot control

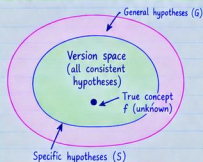
3. Find-S vs. Candidate Elimination

Aspect	Find-S	Candidate Elimination
Learning type	Supervised (concept learning)	Supervised (concept learning)
Maintains	Single most specific hypothesis	Version space (boundaries S and G)
Search direction	General $\rightarrow$ specific	Both specific and general boundaries
Uses negative examples	No	Yes
Result	One hypothesis (may be too specific)	Set of all consistent hypotheses
Optimal?	Not always (may be overfit)	Finds complete version space
Typical use	Simple, fast (initial baseline)	More accurate, handles noise better

4. Concept Learning Workflow



5. Visual Summary: Version Space



S boundary: most specific consistent hypotheses.
G boundary: most general consistent hypotheses.
Version space: all hypotheses between S and G .

6. Common Assumptions

- Training examples are independent and identically distributed (i.i.d.).
- The target function f exists in the hypothesis space H (realizable assumption) (otherwise we get an approximation).
- Training and test data come from the same distribution.
- There is enough training data to learn a good hypothesis.
- Noise may be absent (ideal case) or present (more realistic); algorithms may need modifications to handle noise.

7. Common Exam Questions

- Define: training example, hypothesis, concept, version space, generalization.
- Explain the difference between supervised, unsupervised and reinforcement learning (with examples).
- Trace the Find-S algorithm on a small dataset.
- Trace the Candidate Elimination algorithm (show S and G after each example).
- Given a hypothesis space, determine the final version space.
- Discuss why Find-S may overfit.
- Explain the bias-variance tradeoff.
- Given a learning problem, decide which type of learning is appropriate.
- True/False or short questions on assumptions (e.g., i.i.d., realizable).

8. Common Pitfalls

- Overfitting:** hypothesis fits training data but fails to generalize.
- Underfitting:** hypothesis too simple to capture the true concept.
- Choosing an inappropriate hypothesis space (too restrictive or too large).
- Ignoring negative examples (in Find-S).
- Small or unrepresentative training data.
- Assuming the training and test distributions differ (violates i.i.d.).
- Noise in data can lead to incorrect hypotheses (if not handled properly).

9. Key Takeaways

- Learning learns from experience (observations).
- Supervised, unsupervised and reinforcement learning solve different types of problems.
- A hypothesis space defines what can be learned.
- Find-S is a simple algorithm that finds the most specific consistent hypothesis.
- Candidate Elimination maintains the version space of all consistent hypotheses.
- Generalization is the ability to perform well on new, unseen instances.
- Overfitting is a major issue; prefer simpler hypotheses (Occam's razor).
- Good generalization requires sufficient and representative training data.
- The i.i.d. assumption and realizable assumption are commonly used in analysis.
- Learning from observations is fundamental to building intelligent agents that can adapt to new situations.